

# Phishing Website Detection using Machine Learning

Silin Michael

## Abstract

This project reproduces and critically evaluates a phishing website detection system based on engineered URL and website features. Three classical machine learning models—Logistic Regression, Decision Tree, and Random Forest—were trained and evaluated on a publicly available phishing dataset. The reproduction extends the original work by providing a structured machine learning workflow, comprehensive evaluation metrics, feature engineering analysis, reproducibility analysis, and critical discussion. Among the evaluated models, Random Forest achieved the strongest overall performance, supporting the original claim that engineered features are effective for phishing website detection while also highlighting important limitations regarding real-world deployment and evaluation methodology.

## 1 Summary of the Source

The selected source for this project is the public GitHub repository *Phishing Website Detection*, available at:

<https://github.com/gangeshbaskerr/Phishing-Website-Detection>

The source addresses the problem of phishing website detection. Phishing websites imitate legitimate websites in order to steal sensitive information such as usernames, passwords, financial details, or personal data. This is an important cybersecurity problem because phishing attacks are common, easy to deploy, and can cause serious harm to individuals and organizations.

The proposed solution in the original repository is to use machine learning models trained on engineered website and URL-based features. Instead of analyzing raw web pages directly, the dataset contains extracted features that describe suspicious website behavior and URL structure. The main dataset file used in this project is:

`DataFiles/5.urlldata.csv`

Each row represents one website and contains a binary label:

- **0**: Legitimate website
- **1**: Phishing website

The features include URL-based properties, domain-related properties, and website behavior indicators. Examples include whether the URL contains an IP address, whether it uses a shortened URL, URL length, URL depth, redirection behavior, DNS records, domain age, iframe usage, mouse-over behavior, and right-click behavior.

The original repository demonstrates that classical machine learning models can be used to distinguish phishing websites from legitimate websites. This reproduction investigates whether that claim is supported by the data, whether the methodology is reproducible, and whether the evaluation is sufficiently rigorous.

## 2 Critical Evaluation

The main claim of the original repository is that engineered URL and website-related features can be used with machine learning models to detect phishing websites effectively. My reproduction largely supports this claim, since the trained models achieved strong performance on the held-out test set. In particular, Random Forest achieved the strongest results among the models tested.

However, there are several limitations in the original methodology. First, the original project focuses mainly on model training and performance reporting, but it provides limited critical discussion of the dataset, feature engineering process, and possible weaknesses. A high accuracy value alone is not enough to prove that a phishing detection system is reliable in real-world conditions.

Second, the original work does not sufficiently discuss the consequences of false positives and false negatives. In phishing detection, false negatives are especially dangerous because they represent phishing websites that were incorrectly classified as legitimate. Such errors may allow users to access malicious websites without warning. False positives are also important because they may block legitimate websites, but they are generally less dangerous than failing to detect a real phishing website.

Third, the dataset is approximately balanced between legitimate and phishing websites. This is useful for model training and evaluation, but it may not reflect real-world deployment, where legitimate websites are usually much more common than phishing websites. Therefore, performance on this benchmark dataset may overestimate real-world effectiveness.

Finally, the evaluation relies on a single train-test split. This is acceptable for a basic reproduction, but a stronger evaluation would include cross-validation or testing on a separate external dataset. Overall, the original claim is supported within the limits of the dataset, but the original work would be stronger with more extensive evaluation,

clearer reproducibility instructions, and deeper discussion of feature engineering and error patterns.

### 3 Feature Engineering Analysis

Feature engineering is central to this project. The dataset does not use raw website content directly. Instead, it uses manually engineered features that describe properties of the URL, domain, and website behavior.

The `Domain` column was excluded from model training. This was done because domain names are textual identifiers and could cause the model to memorize specific websites instead of learning general phishing-related patterns. However, the domain names were later used for error analysis, where they helped interpret which websites were misclassified.

The feature matrix  $X$  was created by removing the `Domain` column and the target label column. The target vector  $y$  was created from the `Label` column. The dataset was split into training and testing sets using an 80/20 split, stratified by the target label.

Two preprocessing approaches were used:

- Tree-based models used the original unscaled numeric features.
- Logistic Regression used a `StandardScaler` preprocessing pipeline.

Scaling was used for Logistic Regression because the model is sensitive to feature magnitudes. Standardization transforms a feature  $x$  into:

$$z = \frac{x - \mu}{\sigma}$$

where  $\mu$  is the mean and  $\sigma$  is the standard deviation. This helps Logistic Regression treat features on comparable scales. Tree-based models such as Decision Trees and Random Forests do not require scaling because they split data according to feature thresholds.

From a cybersecurity perspective, many of the engineered features are meaningful. For example, phishing websites often use long URLs, unusual URL depth, shortened links, suspicious redirections, or recently created domains. Features such as `URL_Length`, `URL_Depth`, `TinyURL`, `Domain_Age`, and `DNS_Record` therefore have practical security meaning.

From a mathematical perspective, possible redundancy can be detected using correlation analysis. If two features are highly correlated, they may provide overlapping information. In this project, Spearman correlation was used because many features are binary or ordinal rather than continuous normally distributed variables.

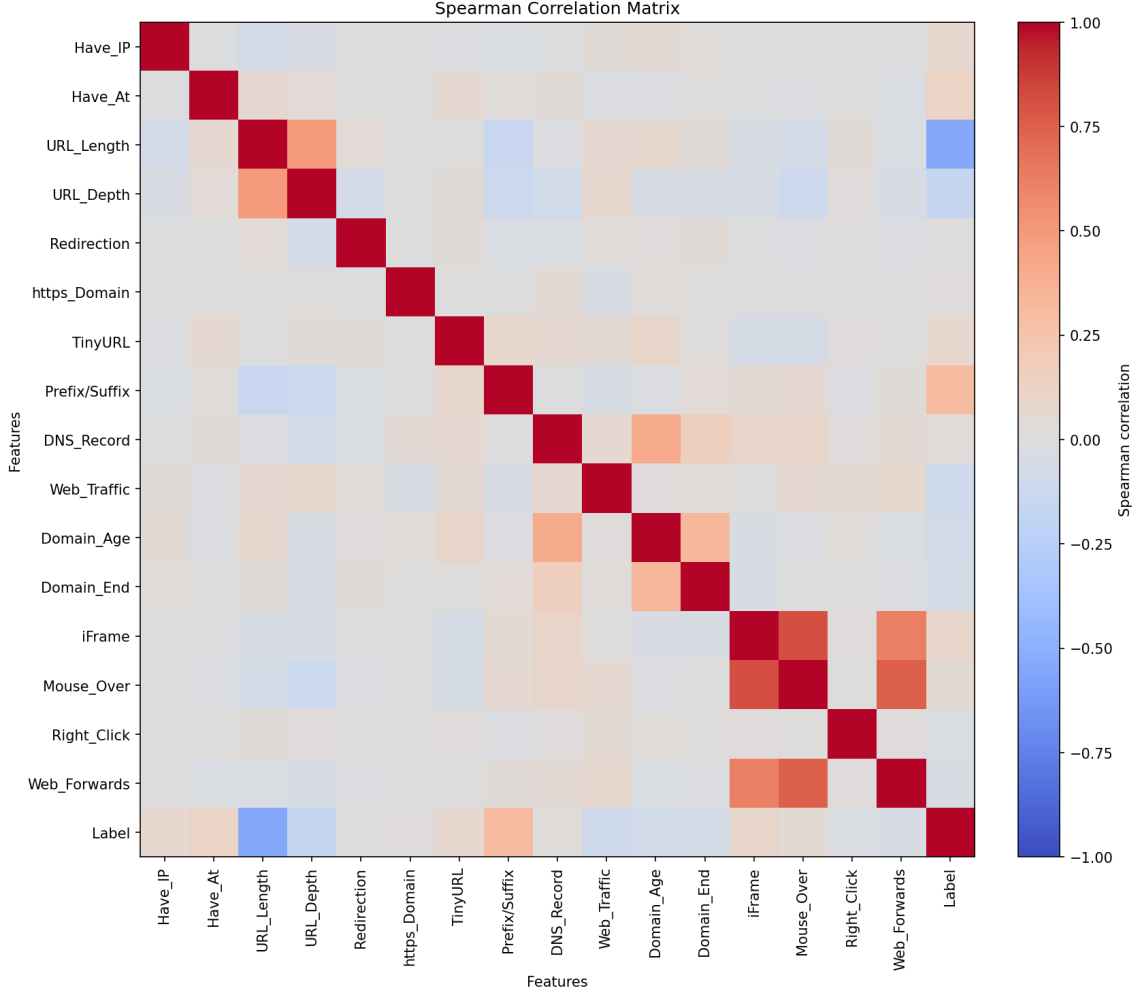


Figure 1: Spearman correlation matrix between engineered features.

Additional features that could improve performance include TLS certificate information, WHOIS registration details, domain reputation scores, character entropy of the URL, page content features, JavaScript behavior, login form detection, and blacklist or threat-intelligence indicators. These features could make the model more robust against modern phishing websites.

## 4 Reproducibility Analysis

The original repository provides the dataset and implementation, which makes reproduction possible. However, the workflow required several adjustments in order to make the project cleaner and more reproducible.

The dataset file `5.urldata.csv` was available in the original repository and could be loaded successfully. The reproduction used a relative path to the original repository:

```
../Phishing-Website-Detection/DataFiles/5.urldata.csv
```

A new notebook was created to reorganize the workflow into a clearer machine learning pipeline. The notebook includes data loading, inspection, EDA, preprocessing, model training, evaluation, feature importance analysis, and error analysis.

Some dependency issues were encountered during reproduction. In particular, XGBoost required the OpenMP runtime on macOS, which introduced an environment-specific issue. To avoid making the notebook dependent on this platform-specific configuration, XGBoost was removed from the final reproduction. The final notebook uses Logistic Regression, Decision Tree, and Random Forest, all of which run successfully with standard scikit-learn dependencies.

The final repository includes a `requirements.txt` file, a README file, supporting source files, generated figures, and the notebook. The notebook uses fixed random seeds where appropriate, which improves reproducibility.

Overall, the original work was reproducible in the sense that the dataset and general approach were available. However, the original repository could be improved by providing clearer dependency instructions, a more structured workflow, and a deeper explanation of preprocessing steps. This reproduction improves reproducibility by organizing the code and documentation more systematically.

## 5 Experimental Results

### 5.1 Exploratory Data Analysis

Exploratory Data Analysis was performed before model training. The class distribution was examined to determine whether the dataset was balanced between phishing and legitimate websites.

A missing-value analysis was also performed before model training. No missing values were found in any feature column or in the target label, so no imputation or row removal was required.

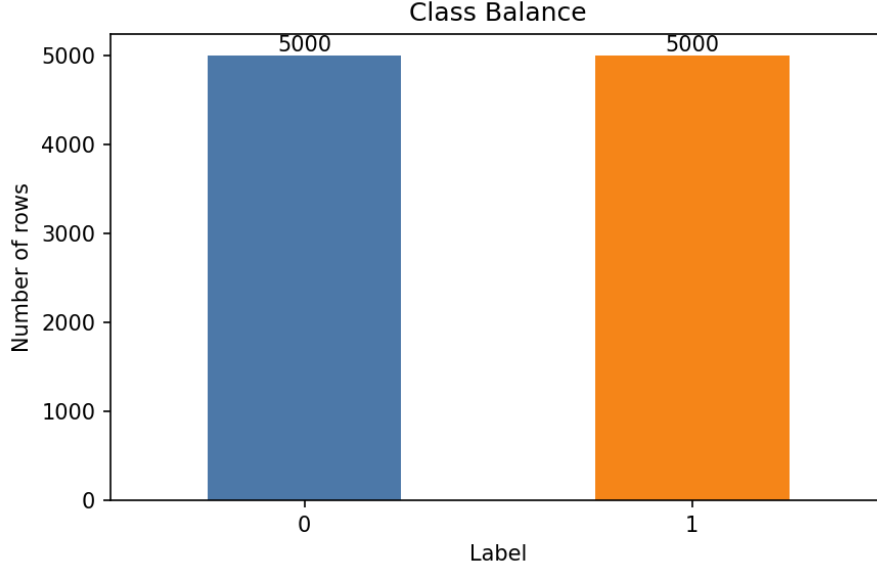


Figure 2: Class balance of legitimate and phishing websites.

The dataset was approximately balanced, meaning that both legitimate and phishing websites were sufficiently represented. Figure 2 illustrates this balanced distribution. Because the two classes contain a similar number of samples, the trained models are less likely to become biased toward one class.

## 5.2 Models Trained

Three classification models were trained and evaluated:

- **Logistic Regression:** A simple linear classification model used as a baseline.
- **Decision Tree:** A tree-based model that learns rule-like decisions from the data.
- **Random Forest:** An ensemble of decision trees that improves stability and generalization.

Each model was trained on the training set and evaluated on the held-out test set.

## 5.3 Evaluation Metrics

The models were evaluated using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and confusion matrices.

Let  $TP$  be true positives,  $TN$  true negatives,  $FP$  false positives, and  $FN$  false negatives.

Accuracy measures the overall proportion of correct predictions:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

In phishing detection, accuracy is useful when the dataset is balanced, but it may hide dangerous false negatives if the dataset is imbalanced.

Precision measures how many websites predicted as phishing were actually phishing:

$$Precision = \frac{TP}{TP + FP}$$

High precision means that when the model raises a phishing alert, the alert is usually correct. This reduces unnecessary blocking of legitimate websites.

Recall measures how many actual phishing websites were detected:

$$Recall = \frac{TP}{TP + FN}$$

Recall is especially important in phishing detection because false negatives allow phishing websites to pass as legitimate.

F1 Score balances precision and recall:

$$F1 = \frac{2 \cdot Precision \cdot Recall}{Precision + Recall}$$

F1 Score is useful when both false positives and false negatives matter.

ROC-AUC (Area Under the Receiver Operating Characteristic Curve) measures the model's ability to distinguish phishing websites from legitimate websites across all possible classification thresholds. Mathematically, it is defined as

$$AUC = \int_0^1 TPR(FPR^{-1}(t)) dt$$

where  $TPR$  is the True Positive Rate and  $FPR$  is the False Positive Rate. A higher ROC-AUC indicates better discrimination between the two classes, independent of a specific decision threshold.

The Matthews Correlation Coefficient (MCC) provides a balanced measure of classification quality by considering all four outcomes of the confusion matrix. It is defined as

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

MCC ranges from  $-1$  to  $1$ , where  $1$  indicates perfect classification,  $0$  corresponds to random prediction, and  $-1$  indicates complete disagreement between predictions and true labels. MCC is particularly useful when evaluating binary classifiers because it remains informative even when class distributions are imbalanced.

The  $F\beta$  Score was not reported separately because this project did not prioritize either precision or recall over the other. Since the dataset is approximately balanced and both false positives and false negatives are important, the F1 Score was selected as the primary

balanced performance metric.

## 5.4 Results

Table 1: Performance of the evaluated machine learning models on the held-out test set.

| Model               | Accuracy | Precision | Recall | F1 Score | ROC-AUC |
|---------------------|----------|-----------|--------|----------|---------|
| Random Forest       | 0.868    | 0.926     | 0.800  | 0.858    | 0.926   |
| Decision Tree       | 0.865    | 0.925     | 0.794  | 0.855    | 0.917   |
| Logistic Regression | 0.797    | 0.923     | 0.647  | 0.761    | 0.874   |

The Random Forest classifier achieved the strongest overall performance on the held-out test set. It obtained an Accuracy of 0.868, Precision of 0.926, Recall of 0.800, F1 Score of 0.858, and ROC-AUC of 0.926. The Decision Tree produced very similar performance, while Logistic Regression achieved the lowest Recall and F1 Score.

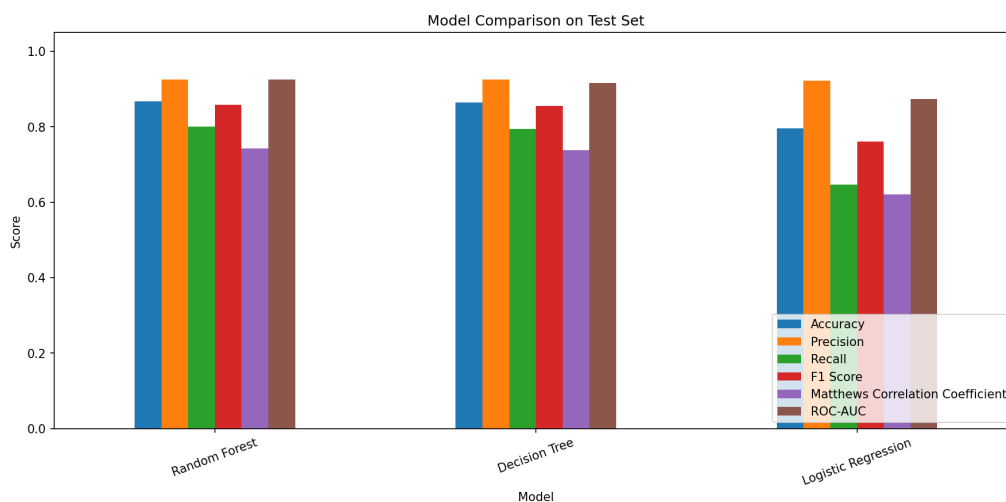


Figure 3: Comparison of model performance across evaluation metrics.

## 5.5 Feature Importance

Feature importance analysis was performed using the Random Forest classifier, which achieved the highest overall performance.



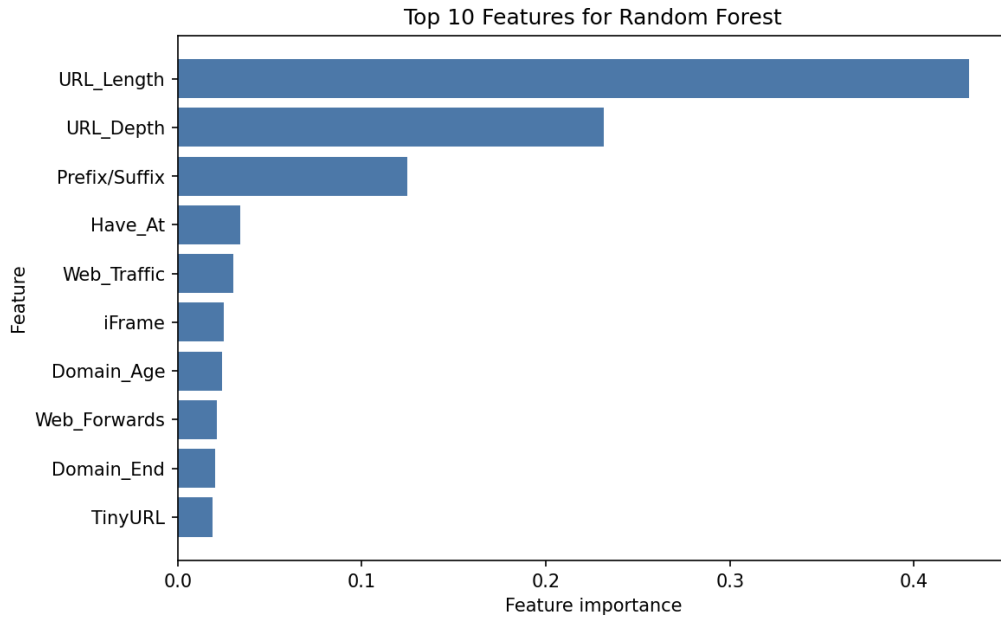


Figure 4: Top feature importance values for the Random Forest model.

As shown in Figure 4, URL Length and URL Depth are among the most influential features. This suggests that structural URL characteristics play an important role in distinguishing phishing websites from legitimate websites.

## 5.6 Error Analysis

Error analysis was performed using the confusion matrices.

Confusion Matrices on Test Set

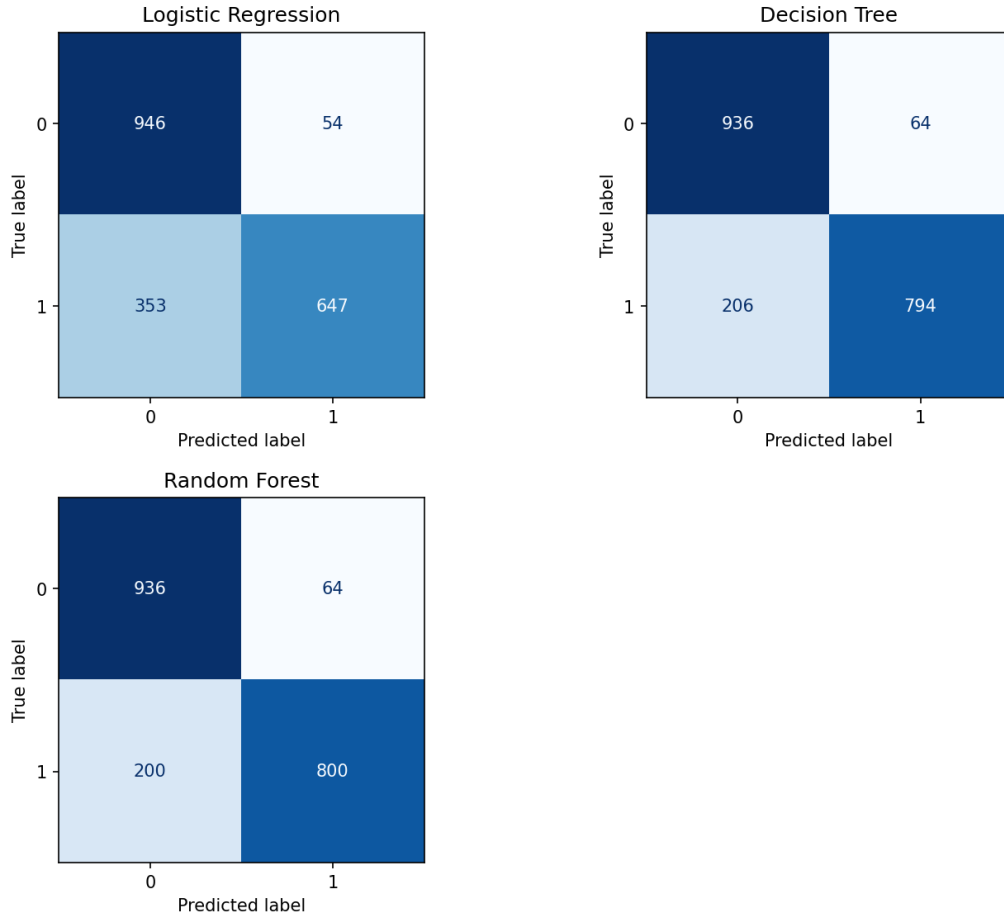


Figure 5: Confusion matrices for the trained models.

For the best-performing Random Forest model, 1,736 websites were classified correctly. The model produced 64 false positives and 200 false negatives. Although the number of false positives was relatively small, the false negatives remain important because phishing websites classified as legitimate may still pose a security risk.

This demonstrates the practical trade-off in phishing detection. A model with high precision avoids falsely blocking legitimate websites, while a model with high recall catches more phishing websites. In cybersecurity, reducing false negatives is often especially important because undetected attacks may cause direct harm.

## 6 Conclusions

This project reproduced and extended a phishing website detection workflow using classical machine learning techniques. The dataset was inspected, analyzed, preprocessed, and used to train Logistic Regression, Decision Tree, and Random Forest classifiers.

The reproduction supports the original author’s general claim that engineered URL

and website-related features can be effective for phishing detection. The Random Forest model achieved the strongest overall performance, with an F1 Score of 0.858 and ROC-AUC of 0.926.

The main strength of the proposed solution is that it uses interpretable engineered features that are meaningful in cybersecurity. Features such as URL length, URL depth, domain age, and redirection behavior reflect properties that are commonly associated with phishing websites.

However, the solution also has weaknesses. The dataset is balanced, while real-world phishing detection is usually imbalanced. The evaluation relies on a single train-test split, and the dataset may not reflect the newest phishing techniques. The system also depends on manually engineered features, which attackers may learn to evade.

Future improvements could include cross-validation, testing on newer datasets, adding richer domain and content-based features, and evaluating more advanced models such as gradient boosting methods or deep learning approaches.

## 7 Executive Summary

This project critically evaluates and reproduces a public phishing website detection repository. The original source proposes using machine learning models trained on engineered URL and website-related features to classify websites as legitimate or phishing.

The problem is important because phishing websites are widely used to steal credentials, financial information, and other sensitive data. Automated detection is necessary because phishing websites can appear quickly and at large scale.

The dataset used in this project is `5.urldata.csv`, taken from the original repository. Each row represents a website and includes engineered features such as URL length, URL depth, use of IP address, shortened URL usage, DNS record information, domain age, iframe behavior, and redirection behavior. The binary label indicates whether the website is legitimate or phishing.

The reproduction trained three classical machine learning models: Logistic Regression, Decision Tree, and Random Forest. The models were evaluated using Accuracy, Precision, Recall, F1 Score, ROC-AUC, and confusion matrices. The Random Forest model achieved the strongest overall performance, with Accuracy of 0.868, Precision of 0.926, Recall of 0.800, F1 Score of 0.858, and ROC-AUC of 0.926.

The reproduction supports the original claim that engineered features can be useful for phishing detection. However, the analysis also shows that the original work has limitations. The original repository provides limited critical evaluation, limited discussion of feature engineering, and insufficient discussion of real-world deployment issues. The dataset is balanced, while real-world phishing detection is usually imbalanced. In addition, evaluation on a single train-test split may not provide a complete estimate of

generalization.

Overall, the project demonstrates that classical machine learning can perform well on the provided phishing dataset, especially when meaningful engineered features are available. However, further validation is required before such a system could be trusted in real-world cybersecurity settings.

## 8 Summing It Up

The selected project addresses phishing website detection using machine learning. The original repository provides a dataset, engineered features, and model implementations for classifying websites as legitimate or phishing.

The reproduction study confirmed that the proposed approach can achieve strong results on the provided dataset. Random Forest performed best among the evaluated models, suggesting that ensemble tree-based methods are well suited to this tabular feature-based phishing detection task.

The author's main claim is mostly supported: engineered URL and website features can help detect phishing websites. However, the strength of this claim is limited by the evaluation methodology. The original work would be stronger if it included cross-validation, more detailed feature engineering analysis, clearer reproducibility instructions, and evaluation on more realistic or recent datasets.

I would recommend using this project as a learning example or baseline for phishing detection. It is useful for understanding how classical machine learning can be applied to cybersecurity data. However, I would not recommend deploying it directly in a real-world security system without additional testing, more recent data, and stronger validation.

The most important insight from this project is that model performance must be interpreted in context. High performance on a benchmark dataset does not automatically imply real-world effectiveness. A successful cybersecurity data science solution must combine predictive performance, interpretability, reproducibility, and critical understanding of the threat environment.

## 9 References

1. Gangesh Basker. *Phishing Website Detection Repository*. <https://github.com/gangeshbaskerr/Phishing-Website-Detection>
2. Scikit-learn Documentation. <https://scikit-learn.org/>
3. Pandas Documentation. <https://pandas.pydata.org/>